

Playwright TypeScript Interview Questions & Answers

Complete Guide for QA Automation Engineers (2025-2026)

 **Prepared by Naveen Automation Labs**

Your Trusted Partner in Test Automation Excellence

 **Learn from the Experts:**

-  Live Training Experience
 -  Real-World Project Insights
 -  Industry Best Practices
 -  Comprehensive Test Automation Solutions
-

Table of Contents

1. Basics & Fundamentals
 2. TypeScript Specific
 3. Locators & Element Interaction
 4. Waits & Auto-waiting
 5. Page Object Model (POM)
 6. Fixtures
 7. API Testing
 8. Network Interception & Mocking
 9. Debugging & Tracing
 10. Configuration & Setup
 11. Parallel Testing & Performance
 12. CI/CD Integration
 13. Advanced Topics
-

Basics & Fundamentals

Q1. What is Playwright and why use it?

Answer: Playwright is an open-source end-to-end testing framework by Microsoft for automating web browsers (Chromium, Firefox, WebKit). It provides a single API for cross-browser testing.

Key Advantages:

- Auto-waiting mechanism (no flaky tests)
- Built-in trace viewer, screenshots, videos
- Supports multiple languages (JS, TS, Python, Java, .NET)
- Headless and headed modes
- Network interception & API testing

Q2. What browsers does Playwright support?

Answer: Chromium, Firefox, and WebKit (Safari). Also works with Chrome, Edge, and Opera.

Q3. How do you install Playwright with TypeScript?

Answer:

```
bash

npm init playwright@latest
# or
npm install -D @playwright/test
npx playwright install
```

Q4. What is the difference between Playwright and Selenium?

Answer:

Feature	Playwright	Selenium
Auto-waiting	Built-in	Manual waits needed
Browser Communication	DevTools Protocol	WebDriver
Speed	Faster	Slower
API Testing	Supported	Not supported
Network Interception	Built-in	Requires additional tools
Installation	Simple	Complex setup

Q5. What programming languages does Playwright support?

Answer: JavaScript, TypeScript, Python, Java, and .NET C#. TypeScript/JavaScript is the most stable and widely used.

TypeScript Specific

Q6. Why use TypeScript with Playwright?

Answer:

- Type safety (compile-time error detection)
- IntelliSense support (better code completion)
- Self-documenting code
- Easier refactoring
- Better tooling support

Q7. How do you configure TypeScript for Playwright?

Answer: Create `tsconfig.json`:

```
json
```

```

{
  "compilerOptions": {
    "target": "ESNext",
    "module": "commonjs",
    "moduleResolution": "node",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  }
}

```

Q8. How do you define types for Page Object Model classes?

Answer:

```

typescript

import { Page, Locator } from '@playwright/test';

export class LoginPage {
  readonly page: Page;
  readonly emailInput: Locator;
  readonly passwordInput: Locator;
  readonly loginButton: Locator;

  constructor(page: Page) {
    this.page = page;
    this.emailInput = page.locator('#email');
    this.passwordInput = page.locator('#password');
    this.loginButton = page.locator('button[type="submit"]');
  }

  async login(email: string, password: string): Promise<void> {
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }
}

```

Locators & Element Interaction

Q9. What are the different locator strategies in Playwright?

Answer:

```
typescript

// CSS Selector
page.locator('.classname')

// XPath
page.locator('xpath=//button[@id="submit"]')

// Text Content
page.locator('text=Login')

// Role-based (Recommended)
page.getByRole('button', { name: 'Login' })

// Label
page.getByLabel('Email')

// Placeholder
page.getByPlaceholder('Enter email')

// Test ID
page.getByTestId('login-button')

// Alt Text
page.getByAltText('Company Logo')
```

Q10. What is the recommended locator strategy?

Answer: Role-based locators are recommended as they align with accessibility standards and are more resilient to UI changes.

```
typescript

await page.getByRole('button', { name: 'Submit' }).click();
```

Q11. How do you interact with elements?

Answer:

```
typescript
```

// Click

```
await page.locator('#submit').click();
```

// Fill text

```
await page.locator('#email').fill('test@example.com');
```

// Type (character by character)

```
await page.locator('#email').type('test@example.com');
```

// Select dropdown

```
await page.locator('#country').selectOption('USA');
```

// Check/Uncheck

```
await page.locator('#agree').check();
```

```
await page.locator('#agree').unchecked();
```

// Double click

```
await page.locator('#element').dblclick();
```

// Right click

```
await page.locator('#element').click({ button: 'right' });
```

// Hover

```
await page.locator('#menu').hover();
```

Q12. What is the difference between fill() and type()?

Answer:

- `fill()`: Clears the field and enters text in one go (faster)
 - `type()`: Types character by character (simulates real user typing)
-

Waits & Auto-waiting

Q13. How does Playwright's auto-waiting work?

Answer: Playwright automatically waits for elements to be:

- Attached to DOM
- Visible
- Enabled
- Stable (not animating) before performing actions. Default timeout: 30 seconds.

Q14. How do you handle explicit waits?

Answer:

```
typescript

// Wait for element to be visible
await page.waitForSelector('#element', { state: 'visible' });

// Wait for network to be idle
await page.waitForLoadState('networkidle');

// Wait for specific element state
await page.locator('#button').waitFor({ state: 'visible' });

// Wait for URL
await page.waitForURL('**/dashboard');

// Wait for response
await page.waitForResponse('**/api/data');

// Custom timeout
await page.click('#submit', { timeout: 5000 });
```

Q15. How do you set different timeout levels?

Answer:

```
typescript

// Test timeout (in playwright.config.ts)
export default defineConfig({
  timeout: 60000, // 60 seconds

  use: {
    navigationTimeout: 30000,
    actionTimeout: 10000
  }
});

// Per action timeout
await page.click('#button', { timeout: 5000 });

// Assertion timeout
await expect(page.locator('#result')).toBeVisible({ timeout: 10000 });
```

Page Object Model (POM)

Q16. What is Page Object Model?

Answer: POM is a design pattern that encapsulates web page elements and their interactions in reusable classes, separating test logic from UI details.

Benefits:

- Improved maintainability
- Code reusability
- Better readability
- Centralized element management

Q17. Implement a complete Page Object example

Answer:

```
typescript
```

```
// pages/LoginPage.ts
```

```
import { Page, Locator, expect } from '@playwright/test';

export class LoginPage {
  readonly page: Page;
  readonly emailInput: Locator;
  readonly passwordInput: Locator;
  readonly loginButton: Locator;
  readonly errorMessage: Locator;

  constructor(page: Page) {
    this.page = page;
    this.emailInput = page.getByLabel('Email');
    this.passwordInput = page.getByLabel('Password');
    this.loginButton = page.getByRole('button', { name: 'Login' });
    this.errorMessage = page.locator('.error-message');
  }

  async goto(): Promise<void> {
    await this.page.goto('/login');
  }

  async login(email: string, password: string): Promise<void> {
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }

  async verifyErrorMessage(expectedMessage: string): Promise<void> {
    await expect(this.errorMessage).toHaveText(expectedMessage);
  }
}
```

```
// tests/login.spec.ts
```

```
import { test, expect } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';

test('Login with valid credentials', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.login('user@example.com', 'password123');
  await expect(page).toHaveURL('/dashboard');
});
```

Fixtures

Q18. What are fixtures in Playwright?

Answer: Fixtures are reusable setup/teardown resources for tests. Playwright provides built-in fixtures like `page`, `context`, `browser`, and `request`.

Q19. How do you create custom fixtures?

Answer:

```
typescript

import { test as base } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';

type MyFixtures = {
  loginPage: LoginPage;
  authenticatedPage: Page;
};

export const test = base.extend<MyFixtures>({
  loginPage: async ({ page }, use) => {
    const loginPage = new LoginPage(page);
    await use(loginPage);
  },

  authenticatedPage: async ({ page }, use) => {
    // Login before each test
    await page.goto('/login');
    await page.fill('#email', 'user@test.com');
    await page.fill('#password', 'password');
    await page.click('#login');
    await use(page);
  }
});

// Usage in test
test('Dashboard test', async ({ authenticatedPage }) => {
  await authenticatedPage.goto('/dashboard');
  // Test already logged in
});
```

Q20. What is fixture scope?

Answer:

- **test-scoped:** Created for each test (default)
- **worker-scoped:** Shared across tests in same worker

typescript

```
export const test = base.extend<{}, { workerStorageState: string }>({  
  workerStorageState: [async ({ browser }, use) => {  
    // Setup once per worker  
    const context = await browser.newContext();  
    // ... login and save state  
    await use(storageState);  
  }, { scope: 'worker' }]  
});
```

API Testing

Q21. How do you perform API testing in Playwright?

Answer:

typescript

```
import { test, expect } from '@playwright/test';

test('API GET request', async ({ request }) => {
  const response = await request.get('https://api.example.com/users');
  expect(response.ok()).toBeTruthy();
  expect(response.status()).toBe(200);

  const data = await response.json();
  expect(data).toHaveLength(10);
});

test('API POST request', async ({ request }) => {
  const response = await request.post('https://api.example.com/users', {
    data: {
      name: 'John Doe',
      email: 'john@example.com'
    }
  });

  expect(response.status()).toBe(201);
  const body = await response.json();
  expect(body.name).toBe('John Doe');
});
```

Q22. How do you handle authentication in API tests?

Answer:

typescript

// Using Bearer Token

```
const api = await request.newContext({
  baseUrl: 'https://api.example.com',
  extraHTTPHeaders: {
    'Authorization': `Bearer ${token}`,
    'Content-Type': 'application/json'
  }
});

const response = await api.get('/protected-resource');
```

// Using Basic Auth

```
const api = await request.newContext({
  httpCredentials: {
    username: 'user',
    password: 'pass'
  }
});
```

Network Interception & Mocking

Q23. How do you intercept network requests?

Answer:

typescript

```

// Log all requests
await page.route('**/*', route => {
  console.log(route.request().url());
  route.continue();
});

// Block specific requests
await page.route('**/*.png,*.jpg,*.jpeg', route => route.abort());

// Mock API response
await page.route('**/api/users', route => {
  route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify([
      { id: 1, name: 'John' },
      { id: 2, name: 'Jane' }
    ])
  });
});

```

Q24. How do you capture network requests and responses?

Answer:

```

typescript

test('Capture network traffic', async ({ page }) => {
  const requests: string[] = [];

  page.on('request', request => {
    requests.push(request.url());
  });

  page.on('response', response => {
    console.log(`${response.status()} ${response.url()}`);
  });

  await page.goto('https://example.com');
});

```

Debugging & Tracing

Q25. How do you debug Playwright tests?

Answer:

typescript

// Method 1: Debug mode

```
npx playwright test --debug
```

// Method 2: page.pause()

```
test('Debug test', async ({ page }) => {  
  await page.goto('https://example.com');  
  await page.pause(); // Opens Playwright Inspector  
  await page.click('#button');  
});
```

// Method 3: Headed mode

```
npx playwright test --headed --slowmo=1000
```

// Method 4: VS Code debugger

// Set breakpoint in VS Code and run in debug mode

Q26. How do you use Playwright Trace Viewer?

Answer:

typescript

```
// In playwright.config.ts
export default defineConfig({
  use: {
    trace: 'on-first-retry', // or 'on', 'retain-on-failure'
    screenshot: 'only-on-failure',
    video: 'retain-on-failure'
  }
});

// Manual tracing
test('Trace example', async ({ page, context }) => {
  await context.tracing.start({
    screenshots: true,
    snapshots: true
  });

  await page.goto('https://example.com');
  // ... test steps

  await context.tracing.stop({ path: 'trace.zip' });
});

// View trace
npx playwright show-trace trace.zip
```

Q27. How do you capture screenshots and videos?

Answer:

```
typescript

// Screenshot
await page.screenshot({ path: 'screenshot.png', fullPage: true });

// Element screenshot
await page.locator('#element').screenshot({ path: 'element.png' });

// Video (in config)
export default defineConfig({
  use: {
    video: 'on', // 'off', 'on-first-retry', 'retain-on-failure'
  }
});
```

Configuration & Setup

Q28. What is playwright.config.ts used for?

Answer: Configuration file to define test settings, browser options, timeouts, reporters, and projects.

```
typescript

import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  timeout: 30000,
  retries: 2,
  workers: 4,

  use: {
    baseURL: 'https://example.com',
    headless: true,
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
    trace: 'on-first-retry',
  },

  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
    },
    {
      name: 'mobile',
      use: { ...devices['iPhone 13'] },
    },
  ],

  reporter: [
    ['html'],
    ['json', { outputFile: 'results.json' }],
    ['junit', { outputFile: 'results.xml' }],
  ],
});
```

Q29. How do you use environment variables?

Answer:

typescript

// .env file

BASE_URL=https://staging.example.com

API_KEY=abc123

// playwright.config.ts

import dotenv **from** 'dotenv';

dotenv.config();

export default defineConfig({

 use: {

 baseUrl: process.env.**BASE_URL**

 }

});

// In test

const apiKey = process.env.**API_KEY**;

Q30. How do you handle multiple environments?

Answer:

typescript

```
// environments/staging.ts
export const config = {
  baseUrl: 'https://staging.example.com',
  apiURL: 'https://api-staging.example.com'
};

// environments/production.ts
export const config = {
  baseUrl: 'https://example.com',
  apiURL: 'https://api.example.com'
};

// playwright.config.ts
import { config } from './environments/staging';

export default defineConfig({
  use: {
    baseUrl: config.baseUrl
  }
});
```

Parallel Testing & Performance

Q31. How does Playwright handle parallel testing?

Answer: Playwright runs tests in parallel by default. Each test runs in an isolated worker process.

```
typescript

// playwright.config.ts
export default defineConfig({
  workers: 4, // Number of parallel workers
  fullyParallel: true, // Run all tests in parallel
});

// Control per test file
test.describe.configure({ mode: 'parallel' });

// Run tests serially
test.describe.configure({ mode: 'serial' });
```

Q32. How do you run tests on multiple browsers?

Answer:

typescript

```
export default defineConfig({
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
  ]
});
```

// Run specific project

```
npx playwright test --project=chromium
```

Q33. How do you handle test dependencies?

Answer:

typescript

```
test.describe.serial('Order matters', () => {
  test('runs first', async ({ page }) => {
    // ...
  });

  test('runs second', async ({ page }) => {
    // Depends on first test
  });
});
```

CI/CD Integration

Q34. How do you run Playwright in CI/CD?

Answer:

yaml

GitHub Actions

name: Playwright Tests

on: [push, pull_request]

jobs:

test:

runs-on: ubuntu-latest

steps:

- **uses:** actions/checkout@v3

- **uses:** actions/setup-node@v3

with:

node-version: 18

- **run:** npm ci

- **run:** npx playwright install --with-deps

- **run:** npx playwright test

- **uses:** actions/upload-artifact@v3

if: always()

with:

name: playwright-report

path: playwright-report/

Q35. How do you handle authentication storage across tests?

Answer:

typescript

```
// global-setup.ts
import { chromium } from '@playwright/test';

async function globalSetup() {
  const browser = await chromium.launch();
  const context = await browser.newContext();
  const page = await context.newPage();

  await page.goto('https://example.com/login');
  await page.fill('#email', 'user@test.com');
  await page.fill('#password', 'password');
  await page.click('#login');

  // Save storage state
  await context.storageState({ path: 'auth.json' });
  await browser.close();
}

export default globalSetup;

// playwright.config.ts
export default defineConfig({
  globalSetup: './global-setup.ts',
  use: {
    storageState: 'auth.json'
  }
});
```

Advanced Topics

Q36. How do you handle iframes?

Answer:

```
typescript
```

```
// Get frame by name
const frame = page.frame('iframe-name');
await frame.click('#button');

// Get frame by URL
const frame = page.frameLocator('iframe[src*="example.com"]');
await frame.locator('#button').click();

// Nested iframes
const childFrame = frame.frameLocator('iframe#child');
await childFrame.locator('#element').click();
```

Q37. How do you handle file uploads?

Answer:

```
typescript

// Single file
await page.setInputFiles('input[type="file"]', 'path/to/file.pdf');

// Multiple files
await page.setInputFiles('input[type="file"]', [
  'file1.pdf',
  'file2.pdf'
]);

// From buffer
await page.setInputFiles('input[type="file"]', {
  name: 'file.txt',
  mimeType: 'text/plain',
  buffer: Buffer.from('file content')
});

// Clear files
await page.setInputFiles('input[type="file"]', []);
```

Q38. How do you handle file downloads?

Answer:

```
typescript
```

```
test('Download file', async ({ page }) => {
  const [download] = await Promise.all([
    page.waitForEvent('download'),
    page.click('text=Download')
  ]);

  // Save to specific path
  await download.saveAs('downloads/file.pdf');

  // Get file path
  const path = await download.path();
  console.log(path);
});
```

Q39. How do you handle multiple tabs/windows?

Answer:

```
typescript

test('Multiple tabs', async ({ context, page }) => {
  // Click link that opens new tab
  const [newPage] = await Promise.all([
    context.waitForEvent('page'),
    page.click('a[target="_blank"]')
  ]);

  await newPage.waitForLoadState();
  console.log(await newPage.title());

  // Switch back to original page
  await page.bringToFront();
});
```

Q40. How do you emulate mobile devices?

Answer:

```
typescript
```

```
import { devices } from '@playwright/test';

test('Mobile test', async ({ browser }) => {
  const context = await browser.newContext({
    ...devices['iPhone 13']
  });

  const page = await context.newPage();
  await page.goto('https://example.com');
});

// In config
export default defineConfig({
  projects: [
    {
      name: 'Mobile Chrome',
      use: { ...devices['Pixel 5'] }
    }
  ]
});
```

Q41. How do you handle browser permissions?

Answer:

```
typescript

const context = await browser.newContext({
  permissions: ['geolocation', 'notifications'],
  geolocation: { latitude: 37.7749, longitude: -122.4194 }
});
```

Q42. How do you handle cookies?

Answer:

```
typescript
```

```
// Get cookies
const cookies = await context.cookies();

// Add cookies
await context.addCookies([
  {
    name: 'session',
    value: 'abc123',
    domain: 'example.com',
    path: '/'
  }
]);

// Clear cookies
await context.clearCookies();
```

Q43. What are soft assertions?

Answer:

```
typescript

import { test, expect } from '@playwright/test';

test('Soft assertions', async ({ page }) => {
  await page.goto('https://example.com');

  // Continues even if fails
  await expect.soft(page.locator('#element1')).toHaveText('Text 1');
  await expect.soft(page.locator('#element2')).toHaveText('Text 2');

  // Test marked as failed at the end
});
```

Q44. How do you use test annotations?

Answer:

```
typescript
```

```
// Skip test
test.skip('Skip this test', async ({ page }) => {
  // ...
});

// Only run this test
test.only('Run only this', async ({ page }) => {
  // ...
});

// Conditional skip
test('Skip on Firefox', async ({ page, browserName }) => {
  test.skip(browserName === 'firefox', 'Not working on Firefox');
  // ...
});

// Mark as failing
test('Known issue', async ({ page }) => {
  test.fail();
  // Test expected to fail
});

// Slow test
test('Slow test', async ({ page }) => {
  test.slow(); // Triples timeout
  // ...
});
```

Q45. How do you create test reports?

Answer:

typescript

```
// playwright.config.ts
export default defineConfig({
  reporter: [
    ['html'],
    ['json', { outputFile: 'results.json' }],
    ['junit', { outputFile: 'results.xml' }],
    ['allure-playwright'],
    ['list']
  ]
});

// View HTML report
npx playwright show-report
```

Bonus Questions

Q46. How do you use Playwright Codegen?

Answer:

```
bash

# Record tests
npx playwright codegen https://example.com

# With specific browser
npx playwright codegen --browser=firefox

# With device emulation
npx playwright codegen --device="iPhone 13"

# Save to file
npx playwright codegen --target=typescript -o tests/example.spec.ts
```

Q47. How do you retry failed tests?

Answer:

```
typescript
```

```
// In config
export default defineConfig({
  retries: 2, // Retry failed tests twice
});

// Per test
test('Flaky test', async ({ page }) => {
  test.slow();
  test.retry(3); // Retry 3 times
});
```

Q48. How do you organize tests with tags?

Answer:

```
typescript

test('@smoke @critical Login test', async ({ page }) => {
  // ...
});

test('@regression Signup test', async ({ page }) => {
  // ...
});

// Run specific tags
// npx playwright test --grep @smoke
// npx playwright test --grep-invert @slow
```

Q49. How do you handle localStorage and sessionStorage?

Answer:

```
typescript
```

```
// Get storage
const storage = await page.evaluate(() => {
  return {
    local: localStorage.getItem('key'),
    session: sessionStorage.getItem('key')
  };
});

// Set storage
await page.evaluate(() => {
  localStorage.setItem('key', 'value');
  sessionStorage.setItem('key', 'value');
});

// Clear storage
await page.evaluate(() => localStorage.clear());
```

Q50. What are the best practices for Playwright testing?

Answer:

- Use TypeScript for type safety
 - Implement Page Object Model
 - Use role-based locators
 - Leverage auto-waiting (avoid manual waits)
 - Use fixtures for reusable setup
 - Enable tracing and screenshots
 - Run tests in parallel
 - Use meaningful test names
 - Keep tests independent and isolated
 - Use data-testid for critical elements
 - Implement proper error handling
 - Use environment variables for configuration
 - Follow DRY principle
 - Write self-documenting code
 - Regular code reviews and refactoring
-

Common Tricky Questions

Q51. How do you handle dynamic elements with changing IDs?

Answer:

```
typescript

// Use partial attribute match
await page.locator('[id^="dynamic-"]').click();

// Use text content
await page.locator('text=Submit').click();

// Use role
await page.getByRole('button', { name: /submit/i }).click();

// Use data attributes
await page.locator('[data-testid="submit-btn"]').click();
```

Q52. How do you wait for an element to disappear?

Answer:

```
typescript

await page.locator('#loading').waitFor({ state: 'hidden' });
// or
await expect(page.locator('#loading')).toBeHidden();
```

Q53. How do you execute JavaScript in Playwright?

Answer:

```
typescript
```

```
// Evaluate in page context
```

```
const result = await page.evaluate(() => {  
  return document.title;  
});
```

```
// Pass arguments
```

```
const text = await page.evaluate(  
  (selector) => document.querySelector(selector).textContent,  
  '#element'  
);
```

```
// Evaluate on element
```

```
const value = await page.locator('#input').evaluate(  
  (el: HTMLInputElement) => el.value  
);
```

About This Document

Prepared By: Naveen Automation Labs

YouTube Channel: Naveen AutomationLabs

Company: Naveen Automation Labs FZCO, Dubai

This comprehensive guide is curated from **live training sessions**, **real-world project experience**, and **industry best practices** delivered by Naveen Automation Labs to thousands of QA professionals worldwide.

About Naveen Automation Labs

- Leading Test Automation Training Platform
- Expert in Selenium, Playwright, Cypress, WebDriverIO, and API Testing
- Providing industry-standard training and tools for the QA community

Connect With Us

- **Website:** <https://naveenautomationlabs.com>
 - **YouTube:** <https://www.youtube.com/@naveenautomationlabs>
 - **LinkedIn:** Follow for daily QA tips and updates
-

Note: This is a comprehensive guide based on real interview experiences and live training data. Focus on understanding concepts rather than memorizing answers. Practice with real projects for best interview preparation.

© 2025-2026 Naveen Automation Labs. All rights reserved.